JURNAL PRAKTIKUM IOT

MODUL 2

KOMUNIKASI WEBSOCKET DENGAN SERVER LOKAL

Kelompok : .....

Hari/Shift : .....

Kode Asisten Pengajar : .....

Nama : .....

NIM : .....

PRAKTIKUM PEMROGRAMAN PERANGKAT IOT

FAKULTAS TEKNIK ELEKTRO

UNIVERSITAS TELKOM

2026

A. Rangkaian dan Sistem

1. Siapkan komponen berikut:

   - ESP32 DevKit V1 (DOIT) (1x)
   - Orange Pi 4 PRO (1x, sebagai server lokal)
   - Sensor DHT22 (1x)
   - Breadboard (1x)
   - Kabel/Jumper (disesuaikan)

2. Buatlah rangkaian berikut menggunakan daftar alat dan komponen yang telah kalian siapkan! (DHT: Sensor DHT22)



Client (ESP32 DevKit V1 (DOIT))		Server (Orange Pi 4 PRO)
Komponen	**Pin**	**Komponen**
DHT (VCC)	3V3	Tanpa rangkaian tambahan. Server WebSocket berjalan sepenuhnya sebagai perangkat lunak, sehingga Orange Pi cukup terhubung ke jaringan Wi-Fi yang sama dengan ESP32.
DHT (DATA)	GPIO16	
DHT (GND)	GND	

Catatan: DHT22 tidak boleh dibaca lebih cepat dari sekali tiap 2 detik; interval yang lebih rapat mengembalikan `NAN`. Modul ini tidak memakai aktuator apa pun: ESP32 hanya membaca dan mengirim, sedangkan Orange Pi hanya mencatat dan membalas dengan waktu terimanya sendiri.

Pembagian path per kelompok. Server membedakan papan tiap kelompok dari path-nya, sehingga beberapa kelompok dapat mengirim ke satu server yang sama tanpa saling mengganggu. Pakailah path milik kelompok kalian saja.

Kelompok	`WS_PATH`	Kelompok	`WS_PATH`
1	`/grup1`	5	`/grup5`
2	`/grup2`	6	`/grup6`
3	`/grup3`	7	`/grup7`
4	`/grup4`	8	`/grup8`

3. Sebelum menjalankan program, siapkan lingkungan kerja kalian dengan urutan berikut. Centanglah setiap langkah yang telah kalian selesaikan!

- ☐ Memasang pustaka `DHT sensor library` dan `Adafruit Unified Sensor` melalui Library Manager Arduino IDE
- ☐ Memasang pustaka `WebSockets` karya Markus Sattler melalui Library Manager Arduino IDE
- ☐ Menyiapkan lingkungan virtual `websockets` pada Orange Pi (lihat kotak di bawah)
- ☐ Mencatat alamat IP Orange Pi dengan `hostname -I`
- ☐ Memakai path milik kelompok kalian pada `WS_PATH` (lihat tabel di atas)
- ☐ Menyamakan nilai port dan path pada kedua berkas program

Kebutuhan paket: modul `websockets.asyncio.server` baru tersedia pada `websockets` versi 13 ke atas, sedangkan paket bawaan sistemnya umumnya masih lebih lama.

```
python3 -m venv ~/venv-iris
source ~/venv-iris/bin/activate
pip install "websockets>=13"
```

4. Lengkapi kode program di bawah ini dengan mengisikan bagian-bagian yang rumpang. Selanjutnya (atau sambil melengkapi), ketiklah kembali bagian-bagian rumpang yang telah diisi pada aplikasi VS Code/Arduino IDE (untuk ESP32) dan editor teks pada Orange Pi (untuk server Python) yang telah disiapkan oleh asisten jaga kalian masing-masing!

   Bagian rumpang ditandai dengan garis bawah `_____` dan diberi nomor `(1)` sampai `(6)` pada komentar di sebelah kanannya. Gantilah setiap garis bawah tersebut dengan kode yang benar sesuai petunjuk pada komentarnya. Perhatikan bahwa nilai **port** dan **path** pada server Python harus sama persis dengan nilai yang dipakai pada kode ESP32.

Server Source Code — WebSocket (Python, Orange Pi)

```
from websockets.asyncio.server import ServerConnection, serve
from websockets.exceptions import ConnectionClosed

HOST: str = "0.0.0.0"  # Mendengarkan pada semua antarmuka jaringan
PORT: int = _________  # (1) Nomor port, sama dengan WS_PORT pada semua kode ESP32

# Path yang diterima, satu per kelompok. Delapan path disediakan; hapus yang
# tidak dipakai atau tambahkan yang baru bila jumlah kelompok berubah.
ALLOWED_PATHS: set[str] = _________  # (2) Path yang diterima, sama dengan WS_PATH tiap ESP32


def timestamp() -> str:
    """Mengembalikan jam dinding saat ini dalam format HH:MM:SS:MS."""
…
            received_at = timestamp()

            # Pesan datang sebagai teks JSON dan harus diubah menjadi dict
            # sebelum nilainya dapat dibaca.
            reading = _________(message)  # (3) Fungsi pengurai teks JSON menjadi dict

            seq = int(reading["seq"])
            temperature = float(reading["temp"])
            humidity = float(reading[_________])  # (4) Kunci JSON kelembapan, sama dengan
pada ESP32

            print(f"[{received_at}] {group} #{seq}: {temperature:.1f} C, "
                  f"{humidity:.1f} %")
…
            # Balasan hanya berisi tanda terima: nomor urut dan waktu terima
            # di sisi server, agar dapat dibandingkan pada tabel jurnal.
            response = json.dumps({"seq": seq, "at": received_at})
            await websocket._________(response)  # (5) Metode pengirim balasan

    except ConnectionClosed:
```

```
        print(f"[{timestamp()}] {group} terputus")
…
async def main() -> None:
    # ping_interval/ping_timeout menutup koneksi yang tidak lagi membalas
    # pong, agar koneksi "zombie" tidak membuat ESP32 menunggu balasan selamanya.
    runner = _________  # (6) Fungsi penjalan server
    async with runner(
        handle_client, HOST, PORT, ping_interval=15, ping_timeout=10
    ):
        print(f"[{timestamp()}] Server WebSocket berjalan pada {HOST}:{PORT}")
```

Client Source Code — WebSocket (ESP32)

```
 * adalah petunjuknya.
 */

#include <WiFi.h>
#include <_________>  // (1) Header pustaka client WebSocket (arduinoWebSockets)
#include <DHT.h>
#include <time.h>
#include <sys/time.h>  // Untuk gettimeofday() beresolusi mikrodetik
…
// Path inilah yang memberi tahu server papan kelompok mana yang menyambung.
// Pakailah path milik kelompok kalian saja: "/grup1", "/grup2", "/grup3",
// "/grup4", "/grup5", "/grup6", "/grup7", atau "/grup8". Nilainya harus ada
// di dalam ALLOWED_PATHS pada server.
const char*    WS_PATH = "_________";  // (2) Path, sama dengan pada server Python

// NTP: penyelaras jam dinding. Offset 25200 detik setara dengan WIB (UTC+7).
const char*  NTP_SERVER     = "pool.ntp.org";
const long   GMT_OFFSET_SEC = 25200;
…
// Menyusun satu bingkai JSON berisi pembacaan DHT22 terbaru. Mengembalikan
// string kosong bila sensornya gagal dibaca.
String buildReading() {
  float humidity    = dht.readHumidity();
  float temperature = dht._________();  // (3) Metode pembaca suhu dalam Celsius

  if (isnan(humidity) || isnan(temperature)) {
    Serial.println("DHT22 gagal dibaca, periksa kabel data dan catu dayanya.");
    return "";
…
  connectToWiFi();
  syncClock();

  webSocket.begin(WS_HOST, WS_PORT, WS_PATH);
  webSocket.onEvent(_________);  // (4) Nama fungsi callback di atas, tanpa tanda kurung

  // Ping tiap 15 detik; setelah 2 kali gagal dibalas dalam 3 detik, papan
  // memutus sendiri koneksi yang macet dan reconnect otomatis berjalan.
  webSocket.enableHeartbeat(15000, 3000, 2);
…
  Serial.println("Pembacaan DHT22 akan dikirim setiap 2 detik.");
}

void loop() {
  webSocket._________();  // (5) Metode pemroses event, wajib dipanggil berkala

  if (!webSocketConnected) {
    return;
  }
…
  sendTime = millis();
  waitingForReply = true;

  webSocket._________(message);  // (6) Metode pengirim pesan teks

  Serial.printf("[%s] Kirim #%lu: %s\n",
                wallClockTimestamp().c_str(), seq, message.c_str());
}
```

5. Catatlah konfigurasi yang kalian pakai pada tabel berikut ini! Ketiga nilai teratas harus sama persis antara berkas server Python dan berkas ESP32, jika tidak maka koneksi WebSocket tidak akan pernah terbentuk.

Parameter	**Nilai yang dipakai**
Alamat IP Orange Pi (`WS_HOST` / `hostname -I`)	
Port server (`PORT` / `WS_PORT`)	
Path WebSocket (`WS_PATH`)	
Nama jaringan Wi-Fi (`WIFI_SSID`)	
Pin data DHT22 pada ESP32 (`DHT_PIN`)	
Jeda pengambilan sampel (`SAMPLE_INTERVAL_MS`)	

B. Analisis

1. Gambarkanlah alur komunikasi yang diterapkan pada sistem yang telah kalian kerjakan!

2. Selanjutnya akan dilakukan percobaan respons waktu pembacaan data yang dikirim melalui **Client (ESP32)** ke **Server (Orange Pi)**. Bukalah terlebih dahulu Serial Monitor pada ESP32 dan terminal server Python pada Orange Pi. Setiap 2 detik ESP32 mencetak **waktu kirim** beserta nomor urut dan pembacaan DHT22, sementara terminal Orange Pi mencetak **waktu terima** beserta nomor urut, suhu, dan kelembapan yang sama. Nomor urut itulah yang dipakai untuk memasangkan baris pada kedua tampilan. Balasan server tidak membawa perintah apa pun, hanya nomor urut dan waktu terima pada kolom `at`, sehingga kedua waktu tetap dapat dibaca ulang dari Serial Monitor ESP32 saja. Catatlah:
   - **LIMA (5)** pasang data: waktu kirim, suhu, dan kelembapan dari Serial Monitor ESP32, beserta waktu terima dari terminal Orange Pi

   Kedua waktu tersebut dicatat dalam format **HH:MM:SS:MS**. Hitunglah **delay satu arah** dengan mengurangkan waktu terima terhadap waktu kirim, lalu bandingkan dengan delay round-trip yang dicetak sendiri oleh ESP32.

   Catatan: ESP32 dan Orange Pi berada pada satu jaringan Wi-Fi, sehingga jam keduanya diselaraskan melalui NTP (`configTime()` pada ESP32, systemd-timesyncd pada Orange Pi). Karena itu waktu dari kedua tampilan benar-benar dapat dikurangkan dan hasilnya merupakan delay satu arah yang

sesungguhnya. Ketelitian NTP di atas Wi-Fi berkisar 10–50 ms, jadi delay satu arah yang kalian hitung wajar bila tidak persis setengah dari delay round-trip.

Percobaan ke-...	Suhu **[°C]**	Kelembapan **[%]**	Waktu kirim *(ESP32)* **[HH:MM:SS:MS]**	Waktu terima *(Orange Pi)* **[HH:MM:SS:MS]**	Delay satu arah **[ms]**	Delay round-trip **[ms]**
1.						
2.						
3.						
4.						
5.						
Rata-rata delay (ms)						

3. Tuliskanlah **analisis** kalian pada sistem yang telah kalian eksperimentasikan sebelumnya pada bagian pengisian berikut ini!

(Berikan juga **hubungan/korelasinya** dengan menerapkan **metodologi komunikasi** yang sedang kalian pelajari pada modul ini; tidak harus penuh!)

C. Saran dan Masukan

1. Asisten

2. Laboratorium